



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

POROČILO DELA

TraceBit

Luka Majstorović & Izidor Robnik

Kazalo

Organizacija in vodenje dela	1
Sprint I	2
➤ Struktura baze (ER Model, Tabele)	2
➤ Oblikovanje začetnega vmesnika z osnovno formo za zajem podatkov	2
➤ Fetch zajetih podatkov in pošiljanje na backend	2
➤ Implementacija TypeScript zbiranja podatkov	3
➤ Implementacija prvega API endpointa (POST za sprejem fingerprint podatkov) ...	3
Sprint II	4
➤ Generacija unikatnega hash odtisa	4
➤ Ureditev FrontEnda	4
➤ Deploy projekta na RENDER (Backend in Frontend)	4
➤ Shramba zbranih podatkov v Supabase (PostgreSQL)	5
Sprint III.....	6
➤ Sprememba strukture podatkovne baze	6
➤ Fingerprint napake	6
➤ Implementacija admin uporabniškega panela (dashboard).....	6
➤ Prikaz statistike in fingerprinta v admin panelu	7
➤ Kreiranje logike za uporabnike.....	7
➤ Admin panel – Dostop	7
Sprint IV	8
➤ User Fingerprint (Uporabniški dostop in funkcionalnosti)	8
➤ UI izboljšave (Uporabniška izkušnja)	8
➤ Bugs in sinhronizacija	8
➤ Implementacija detekcije sumljivih fingerprintov z uporabo ML	8
➤ Fingerprint spremembe	9
➤ Implementacija triggerjev za shranjevanje logov aplikacije	9

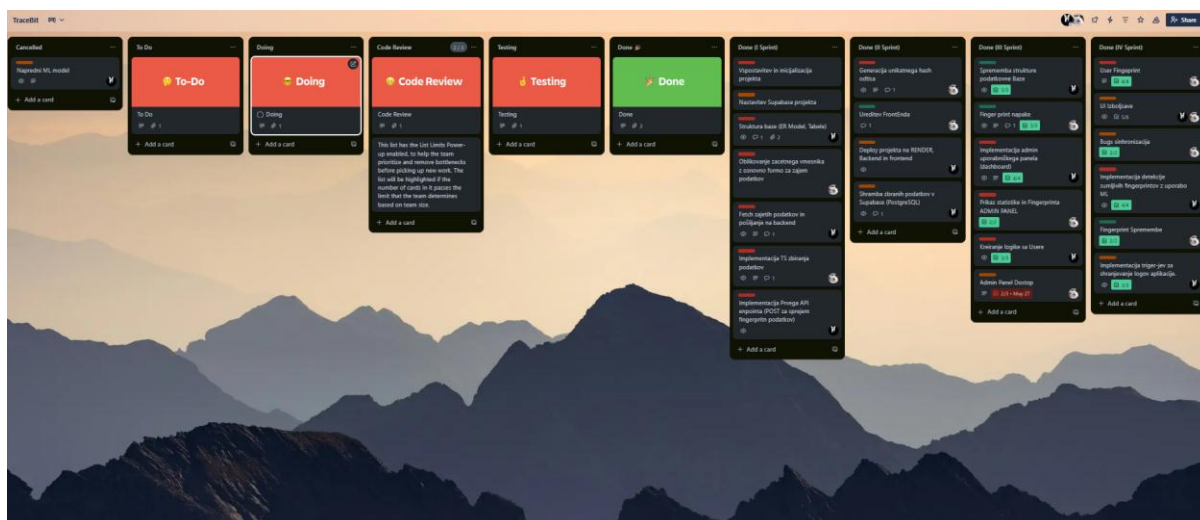
Organizacija in vodenje dela.

Projekt smo izvedli v skladu z agilnim pristopom, ki nam je omogočal sprotno prilagajanje zahtevam, razmeram in odkritim težavam med razvojem. Za vodenje nalog in sprintov smo uporabljali brezplačno orodje **Trello** podjetja Atlassian, ki je zaradi svoje preprostosti in preglednosti omogočalo učinkovito in enostavno organizacijo dela brez potrebe po kompleksnem vzpostavljanju in povezovanju z razvojnimi orodji.

Razvoj smo razdelili na **štiri sprinte**, vsak v trajanju enega tedna. Posamezne naloge (taski) so bile razporejene glede na njihovo zahtevnost in pomembnost za nadaljnji potek projekta. Pri razporejanju dela smo upoštevali kompetence članov ekipe, vsak je prevzel tiste naloge, na katerih ima največ izkušenj ali znanja, obenem pa smo skušali ohraniti ravnovesje pri obremenjenosti posameznikov. Pomembno vlogo je imelo tudi izražanje interesov posameznikov za določena področja.

Na začetku vsakega sprinta smo izvedli **sestane**, kjer smo pregledali uspešnost preteklega dela – kaj je bilo dobro opravljeno, kaj je bilo potrebno prenesti naprej, in kakšni so cilji prihajajočega tedna. Ta pristop nam je omogočil redno refleksijo nad procesom, sprotno optimizacijo in zmanjšanje napak, ki bi sicer nastale zaradi napačnega razumevanja ciljev ali prioritiet.

Tako organizirano delo nam je omogočilo, da smo **čim bolj optimalno izkoristili čas**, se izognili nepotrebnemu raziskovanju in popravkom, ter gradili stabilen in funkcionalen sistem znotraj načrtanih rokov.



Sprint I

➤ Struktura baze (ER Model, Tabele)

V tej fazi smo zasnovali celotno strukturo baze podatkov v Supabase, ki temelji na PostgreSQL. Oblikovali smo ER model z entitetami, kot so users, fingerprints, loge, in definirali relacije med njimi. Določili smo tipe podatkov, ključe in omejitve za zagotavljanje podatkovne integritete ter optimizacijo poizvedb. Pri snovanju smo upoštevali možnost razširljivosti za nadaljnji razvoj sistema. Tabele so bile ustvarjene neposredno v Supabase UI (SQL editor), obenem pa so bile preverjene tudi možnosti za kasnejšo razširitev baze.

➤ Oblikovanje začetnega vmesnika z osnovno formo za zajem podatkov

Zasnovali smo začetni uporabniški vmesnik z uporabo Next.js in Tailwind CSS, ki omogoča zajem podatkov iz brskalnika. Ustvarili smo osnovno formo, ki omogoča interakcijo z uporabnikom in pripravo podatkov za pošiljanje na strežnik. Poskrbeli smo, da je bil vmesnik modularen, odziven in prilagojen enostavni uporabi. Zasnova temelji na principih dobre uporabniške izkušnje in kasnejše nadgradljivosti.

➤ Fetch zajetih podatkov in pošiljanje na backend

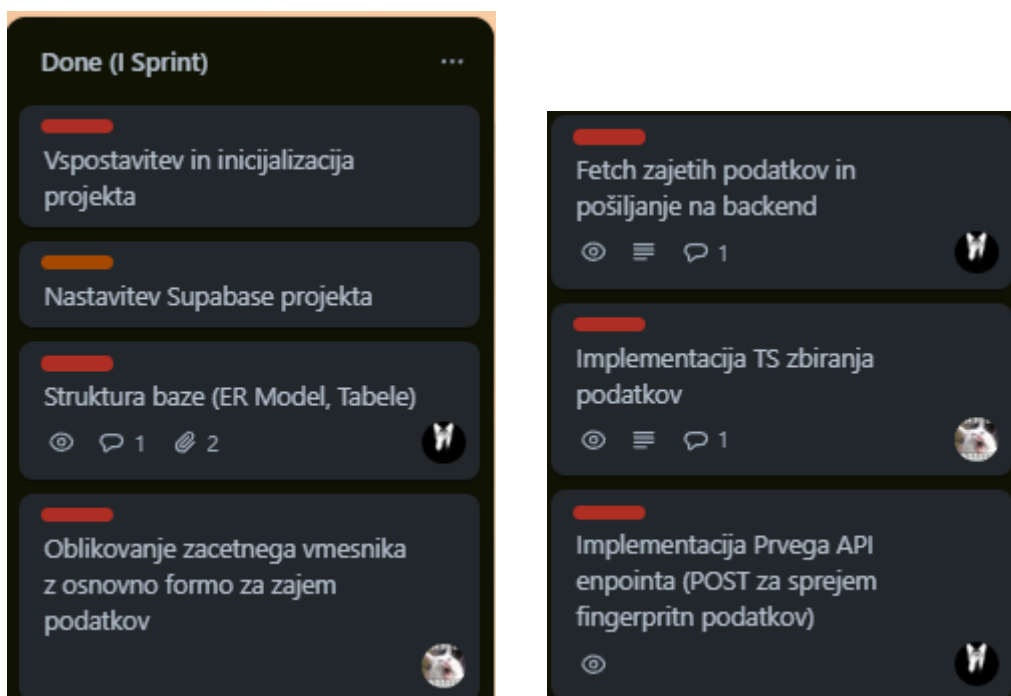
Implementirali smo funkcionalnost za pridobivanje zajetih podatkov in njihovo pošiljanje na backend z metodo POST. Podatke, kot so podatki o napravi, brskalniku in WebGL, smo pridobili prek asinhronne funkcije in jih poslali na API endpoint. Poskrbeli smo za preverjanje pravilnosti prenosa in obravnavo morebitnih napak pri pošiljanju. Funkcionalnost smo tudi testirali s testnimi podatki za zagotavljanje zanesljive komunikacije.

➤ Implementacija TypeScript zbiranja podatkov

S pomočjo TypeScripta smo razvili logiko za zajem podatkov o uporabnikovi napravi in okolju. Pripravili smo funkcije za zbiranje podatkov, kot so jezik sistema, časovni pas, resolucija zaslona, podpora za dotik, in WebGL informacije. Zgradili smo modularno strukturo, ki omogoča enostavno dodajanje dodatnih parametrov v prihodnosti. Kodo smo strukturirali tako, da omogoča ponovno uporabo in enostavno testiranje.

➤ Implementacija prvega API endpointa (POST za sprejem fingerprint podatkov)

Na backend strani smo implementirali prvi API endpoint, ki sprejme fingerprint podatke iz frontend aplikacije. Endpoint smo pripravili z uporabo FastAPI in ga povezali s Supabase bazo, kjer se podatki shranjujejo v ustrezno tabelo. Dodali smo validacijo vhodnih podatkov, logiko za zaznavanje napak in ustrezne odzive. Funkcionalnost smo preverili z dejanskimi zahtevki iz frontend dela in uspešno povezali celotno verigo pošiljanja podatkov.



Sprint II

➤ Generacija unikatnega hash odtisa

V tem koraku smo razvili funkcionalnost za generacijo unikatnega identifikatorja (hash odtisa) na podlagi zbranih podatkov. Uporabili smo funkcijo, ki iz podatkov o napravi in brskalniku ustvari determinističen hash s pomočjo knjižnice crypto (SHA-256). S tem smo zagotovili, da lahko primerjamo fingerprint podatke različnih uporabnikov in zaznamo njihove podobnosti ali razlike. Hash je ključni element za analizo unikatnosti in osnovo za kasnejše zaznavanje anomalij.

➤ Ureditev FrontEnda

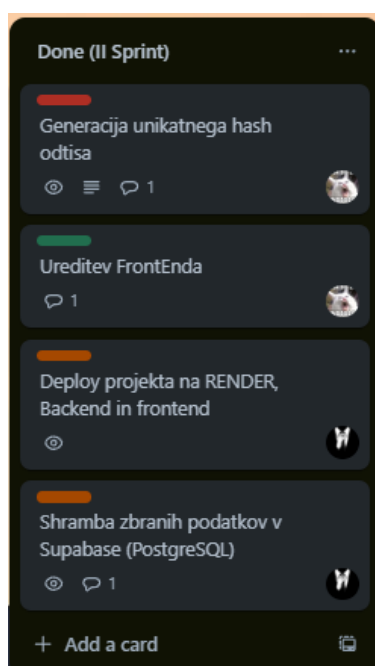
V tej nalogi smo izboljšali obstoječi uporabniški vmesnik in odstranili napake, ki so se pojavile pri prikazu ali interakciji. Preoblikovali smo komponente tako, da so bolj pregledne in enostavne za uporabo ter da omogočajo boljšo odzivnost. Dodatno smo poskrbeli za angleški jezik vmesnika, konsistentno poimenovanje komponent in izboljšano uporabniško izkušnjo. Pripravili smo tudi osnovo za nadaljnjo delitev vmesnika glede na vlogo uporabnika.

➤ Deploy projekta na RENDER (Backend in Frontend)

Projekt smo uspešno postavili na produkcijsko okolje – frontend in backend na Render. Pripravili smo ustrezne konfiguracijske datoteke in okoljske spremenljivke (ENV), da se lahko frontend poveže z backend API-jem. Poskrbeli smo za CORS nastavitve, delovanje JWT avtentikacije in varno komunikacijo med strežnikoma. Po deployu smo opravili testiranje delovanja sistema v realnem okolju in potrdili pravilno delovanje funkcionalnosti.

➤ Shramba zbranih podatkov v Supabase (PostgreSQL)

Vzpostavili smo logiko za shranjevanje zbranih fingerprint podatkov v Supabase bazo. Ob klicu API-ja se vsi podatki najprej validirajo, nato pa shranijo v ustrezno tabelo fingerprints. Poskrbeli smo za varno obdelavo podatkov, uporabo unikatnih ključev (hash) in ločevanje podatkov po uporabniku (prek JWT). Ta korak je bil ključen za začetek shranjevanja in kasnejše analiziranje zbranih podatkov.



Sprint III

➤ Sprememba strukture podatkovne baze

V tem tasku smo spremenili strukturo dveh ključnih tabel v Supabase, users in fingerprints. Dodali smo manjkajoče lastnosti za potrebe poznejših analiz in logiranja, kot so UUID povezave, timestampi in oznake. Dodatno smo ustvarili neposredno povezavo med uporabnikom in njegovimi fingerprinti z uporabo tujega ključa (user_id v tabeli fingerprints). S tem smo zagotovili osnovo za ločevanje podatkov po uporabnikih in omogočili zanesljivo preverjanje avtorstva fingerprintov.

➤ Fingerprint napake

Preverili smo pravilnost zajetih podatkov, pri čemer smo naleteli na konkretno težavo pri zaznavi OS-a, vedno se je shranjevalo "Windows 10", ne glede na dejanski operacijski sistem. Analizirali smo logiko zajema podatkov in ugotovili, da je potrebno natančneje zaznati navigator.userAgent ter dodatno filtrirati verzijo brskalnika. Vendar pa smo ugotovili, da je bila napaka delno posledica tudi omejitev, ki jih postavljajo moderne politike zasebnosti (npr. User-Agent Reduction), kar vpliva tudi na pridobivanje podatkov o CPU in verziji sistema. Preverili smo, ali se beležijo vse ključne informacije, kot so CPU, OS, resolucija ipd., ter s popravki zagotovili čimbolj natančen zajem fingerprint informacij, v okviru tehničnih in pravnih omejitev.

➤ Implementacija admin uporabniškega panela (dashboard)

Zasnovali smo layout za administratorski panel z jasno strukturo, ki vključuje tri glavne sekcije: statistika, pregled fingerprintov, graf anomalij. Uporabili smo zaščito dostopa do admin panela, kjer lahko vstopa le uporabnik z ustrežno vlogo. Na vrh dashboarda smo vključili ključne metrike, kot so skupno število fingerprintov, število unikatnih hashov in zaznanih anomalij. Panel je pripravljen za nadaljnjo razširitev, vključno z integracijo Analiz in dodatnih administratorskih funkcionalnosti.

➤ Prikaz statistike in fingerprinta v admin panelu

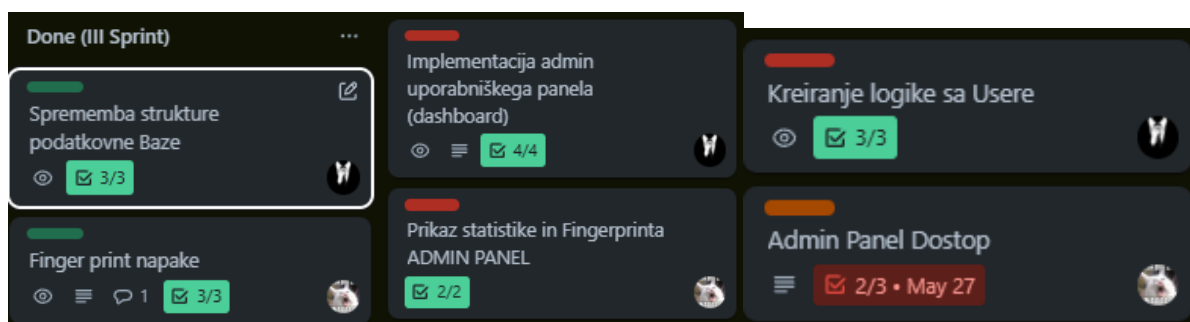
V admin panelu smo prikazali statistične podatke, kot so skupno število zajetih fingerprintov in njihova razčlenitev po značilnostih. Omogočili smo pregled vseh zbranih zapisov iz baze Supabase v obliki tabele z možnostjo iskanja. Dodali smo osnovne statistike: razmerje unikatnih vs. podvojenih fingerprintov. Ta funkcionalnost nudi skrbnikom hiter vpogled v stanje baze in omogoča zgodnje zaznavanje anomalij.

➤ Kreiranje logike za uporabnike

Vzpostavili smo celoten sistem uporabniškega upravljanja, ki vključuje registracijo, prijavo ter ponastavitev gesla. Uporabili smo 2FA (dvofaktorsko avtentikacijo) in email potrditev pri registraciji, kar povečuje varnost aplikacije. Implementirali smo ločevanje uporabnikov po vlogah: "user" in "admin", s preverjanjem JWT žetona za zaščito endpointov. Funkcionalnosti omogočajo zanesljivo in varno interakcijo uporabnikov s sistemom.

➤ Admin panel – Dostop

Zagotovili smo, da je admin panel dostopen samo uporabnikom z vlogo "admin", ki je nastavljena znotraj Supabase baze. Na backendu smo implementirali preverjanje JWT žetona, kjer se preveri, ali ima prijavljen uporabnik pravico za dostop. Na frontend strani pa smo poskrbeli, da se nepravilno avtentificirani uporabniki preusmerijo. Ta mehanizem omogoča varno ločevanje med uporabniškim in administratorskim delom sistema.



Sprint IV

➤ User Fingerprint (Uporabniški dostop in funkcionalnosti)

V tej nalogi smo za prijavljene uporabnike pripravili dva zavihka: ustvarjanje novega fingerprinta, ogled zgodovine, ter urejanje profila. Na prvem zavihku lahko uporabnik z enim klikom zajame nov digitalni odtis, ob tem pa vidi novice ali obvestila. Na drugem zavihku smo omogočili varen in filtriran prikaz preteklih fingerprintov – uporabnik vidi le svoje zapise, admin pa vse. V nastavitvah profila lahko uporabnik nastavi, ali se fingerprinti pošiljajo samodejno ali ob potrditvi, ter ureja geslo, e-pošto in druge nastavitve.

➤ UI izboljšave (Uporabniška izkušnja)

V tem koraku smo izvedli več vizualnih in uporabniških izboljšav. Odpravili smo neželeno premikanje strani (scroll), odstranili neuporabne gumbe in izboljšali prikaz uporabniškega imena po prijavi. Funkcija logout je bila premaknjena pod ikono uporabnika za bolj intuitivno uporabo. Dodali smo fiksni header in poskrbeli, da se admin ob prijavi samodejno preusmeri na admin panel tab.

➤ Bugs in sinhronizacija

Odpravili smo dve napaki, ki sta vplivali na pravilno delovanje uporabniškega vmesnika. Najprej smo popravili napačne povezave, ki so vodile na nepravilne URL in pošiljale podatke na napačne api endpointe. Druga težava se je pojavila pri prijavi admina, njegova prijava se ni vsakic preusmerjala na admin panel. Opravili smo sinhronizacijo z JWT validacijo in poskrbeli za takojšen prikaz vmesnika po prijavi.

➤ Implementacija detekcije sumljivih fingerprintov z uporabo ML

Razvili smo sistem za osnovno in napredno zaznavanje sumljivih fingerprintov z uporabo strojnega učenja. Za neprijavljene uporabnike smo implementirali osnovno logiko zaznave, ki temelji na podobnosti z že obstoječimi odtisi. Pri prijavljenih uporabnikih smo uporabili dodatne kriterije, kot so sprememba naprave, lokacije ali časovni vzorci. Implementirali smo tudi shranjevanje zaznanih sumljivih primerov in ustvarili poseben API endpoint za dostop do teh podatkov.

➤ Fingerprint spremembe

Posodobili smo prikaz fingerprint podatkov tako za admin kot tudi za uporabniško stran. Admin vidi celotno strukturo podatkov, medtem ko uporabnik vidi samo svoje in v bolj poenostavljeni obliki. V primeru zaznanega sumljivega fingerprinta se prikaže dodatno obvestilo na uporabniškem vmesniku. S tem smo povečali transparentnost in obveščenost uporabnikov o morebitnih varnostnih tveganjih.

➤ Implementacija triggerjev za shranjevanje logov aplikacije

V tej fazi smo odstranili neuporabljene tabele in refaktorirali strukturo obstoječih log tabel. Dodali smo triggerje za zajem aktivnosti uporabnikov, vključno s fingerprint logi in prijavi adminov. S tem smo dosegli boljšo sledljivost dogodkov in omogočili revizijo pomembnih varnostnih in sistemskih interakcij. Triggerji so optimizirani tako, da ne vplivajo na zmogljivost aplikacije.

